



University of Applied Sciences

BIT 3.1

Database Systems

08. SQL Queries (1) — Exercises

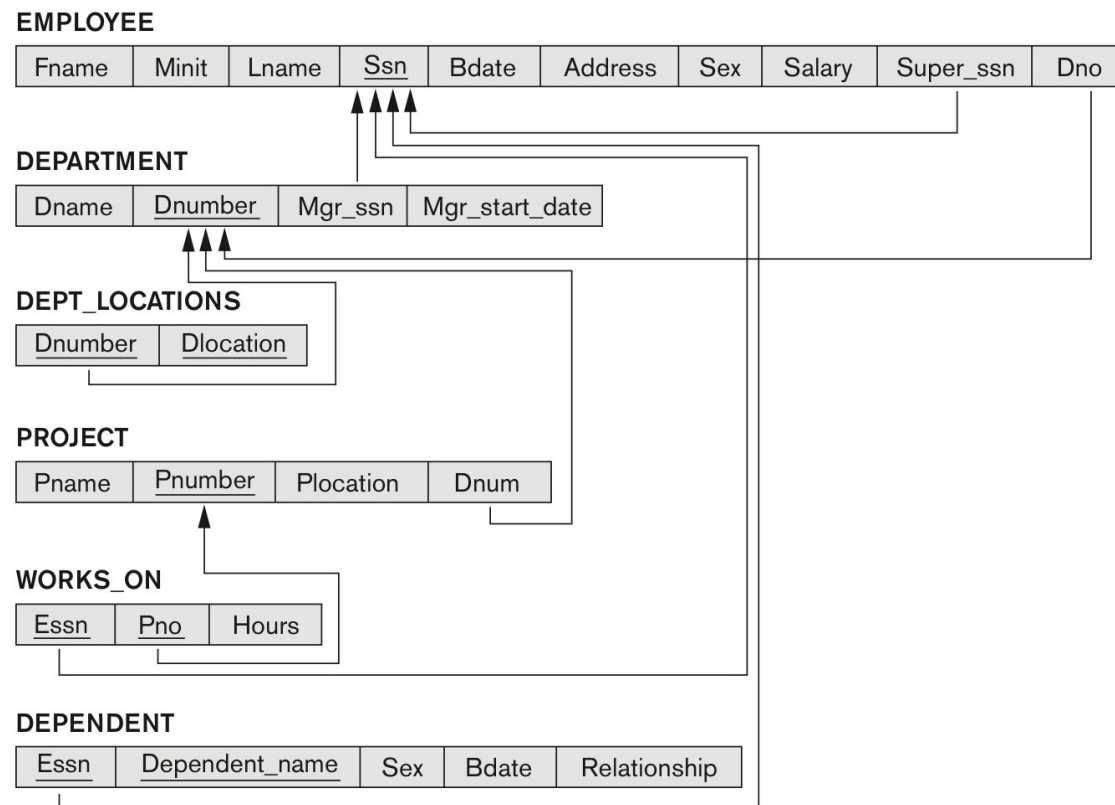
Ibrahim Musabeh

10.04.26

About these exercises

- **21 exercises** covering DML and both SQL Queries lectures.
- **Schema:** the COMPANY database (next slide for a reminder).
- **Tools:** any online SQL editor, or the docker lab provided.
- **Time:** plan around 2 hours total.
- **Organisation:**
 - Part 1 — DML: INSERT, UPDATE, DELETE (4 exercises)
 - Part 2 — Basic SELECT queries (8 exercises)
 - Part 3 — Multi-table queries (4 exercises)

COMPANY relational database schema



A possible database state – COMPANY

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

Part 1 – Data Manipulation (DML)

- INSERT, UPDATE and DELETE statements.
- 4 exercises. Each should take 5–8 minutes.

Watch out for referential integrity when deleting or updating keys.

Exercise 1.1: INSERT

- **Task:** Write an INSERT statement for each scenario.
- a) Add a new employee: Maria Lopez, born 1990-03-15, lives at '500 Main, Houston, TX', female, salary 32000, supervised by Franklin Wong (333445555), department 5.
- b) Add a new project: 'WebRevamp', Pnumber 40, located in 'Sugarland', controlled by department 5.
- c) Add a new dependent for John Smith: 'Lily', female, born 2010-08-22, relationship 'Daughter'.

Remember column order, or list the columns explicitly. (7 min)

Exercise 1.2: UPDATE

- **Task:** Write an UPDATE statement for each scenario.
- a) Give every employee in department 5 a 10% salary raise.
- b) Change Jennifer Wallace's address to '500 Park Ave, Bellaire, TX'.
- c) Change the manager of department 4 to Ahmad Jabbar (Ssn 987987987), with start date 2026-01-15.

Don't forget WHERE — without it, every row is updated! (7 min)

Exercise 1.3: DELETE

- **Task:** Write a DELETE statement for each scenario.
- a) Delete all employees with salary less than 25000.
- b) Delete the project named 'Newbenefits'.
- c) Delete every dependent whose relationship is 'Son'.

DELETE without WHERE empties the entire table. (5 min)

Exercise 1.4: Mixed DML scenario

- **Task:** A new employee, Carla Smith, joins TechCorp. Write the SQL in order.
 - a) INSERT Carla into EMPLOYEE (choose realistic values).
 - b) UPDATE one existing employee so Carla becomes their supervisor.
 - c) DELETE the WORKS_ON rows linked to project Pnumber=30, then DELETE the project itself.
- **Think:** why does the order of (c) matter?

Referential integrity: cannot delete a parent row while children reference it. (10 min)

Part 2 — Basic SELECT queries

- Single-table SELECT with WHERE, ORDER BY, DISTINCT, UNION.
- 8 exercises. Each should take 3–7 minutes.

From SQL Queries (1). If stuck, review the SELECT-FROM-WHERE slide.

Exercise 2.1: Basic WHERE

- **Task:** Write a SELECT query for each scenario.
- a) First and last name of every employee in department 5.
- b) SSN and salary of employees earning more than 30000.
- c) Address of the employee whose SSN is '123456789'.

Operators you may need: = , > , < . (5 min)

Exercise 2.2: Combining conditions with AND / OR

- **Task:** Use AND and OR to combine WHERE conditions.
- a) Employees in department 5 AND earning more than 30000.
- b) Employees in department 4 OR department 1.
- c) Female employees (Sex = 'F') with salary greater than 25000.

AND requires both conditions true. OR requires either one. (5 min)

Exercise 2.3: Predict the output

- **Task:** Predict which rows are returned, then run to check.

```
SELECT Fname, Lname, Salary
FROM   EMPLOYEE
WHERE  Dno = 5 AND Salary >= 38000;
```

- Write down:
 - How many rows are returned?
 - Which employees by name?

Read SQL before writing it. (4 min)

Exercise 2.4: Pattern matching with LIKE

- **Task:** Use LIKE to find employees by partial information.
- a) Employees whose address contains 'Houston'.
- b) Employees whose last name ends with 'g'.
- c) Employees whose first name has exactly 4 letters.

% matches any number of characters; _ matches exactly one. (5 min)

Exercise 2.5: Ranges with BETWEEN

- **Task:** Use BETWEEN to filter by ranges.
- a) Employees with salary BETWEEN 25000 AND 35000.
- b) Employees born BETWEEN '1960-01-01' AND '1970-12-31'.
- c) Projects with Pnumber BETWEEN 10 AND 30.

BETWEEN is inclusive. Small bound first! (5 min)

Exercise 2.6: Find the bug

- **Task:** Each query has a bug. Identify and fix.

a) Goal: distinct salaries in EMPLOYEE

```
SELECT Salary, DISTINCT FROM EMPLOYEE;
```

b) Goal: employees with salary between 25000 and 50000

```
SELECT * FROM EMPLOYEE WHERE Salary BETWEEN 50000 AND 25000;
```

c) Goal: employees whose first name starts with 'J'

```
SELECT * FROM EMPLOYEE WHERE Fname LIKE 'J';
```

Think: DISTINCT placement, BETWEEN bounds, LIKE wildcards. (7 min)

Exercise 2.7: Sorting with ORDER BY

- **Task:** Add ORDER BY to sort query results.
- a) All employees sorted by salary, highest first.
- b) All employees sorted by department (Dno) ASC, then by salary DESC.
- c) All projects sorted by location ASC, then by project name ASC.

Default is ASC. DESC reverses. (5 min)

Part 3 – Multi-table queries

- Joins across two or more tables.
- 4 exercises. Each should take 7–10 minutes.

Tables in FROM, join condition in WHERE, use aliases for clarity.

Exercise 3.1: Two-table join

- **Task:** Combine two tables to answer each question.
- a) For each employee, list their first name, last name and department name.
- b) Name and location of every project controlled by the 'Research' department.
- c) Names of employees who have a dependent named 'Alice'.

Missing join condition = cross product! (8 min)

Exercise 3.2: Three-table join

- **Task:** Combine three tables with WHERE filtering.
- a) For every project in 'Stafford', list:
 - project number, project name
 - controlling department name
 - first and last name of that department's manager
- b) For each employee, list their name, department name, and project name(s) they work on.

Three tables = three join conditions. Use aliases. (10 min)

Exercise 3.3: Self-join (supervisor)

- **Task:** Use the same table twice with aliases.
- a) For each employee, list their first/last name and their supervisor's first/last name.

- **Hint:**

```
FROM EMPLOYEE AS E, EMPLOYEE AS S  
WHERE E.Super_ssn = S.Ssn
```

- b) Adapt your query so employees with no supervisor are clearly indicated.

Two aliases on the same table behave like two different tables. (8 min)

Exercise 3.4: Challenge – mix everything

- **Task:** Write one query that uses several features together.
- List first name, last name and department name of every employee who:
 - earns BETWEEN 25000 and 40000, AND
 - works on at least one project located in 'Houston'.
- Sort by department name ASC, then salary DESC.
- **Features:** multi-table join, BETWEEN, ORDER BY multi-column.

This is what a real business query looks like. (10 min)

Part 4 – Aggregates and subqueries

- COUNT, SUM, AVG, GROUP BY, HAVING, nested queries.
- 5 exercises. Each should take 5–10 minutes.

From SQL Queries (2). Answers questions about groups of rows, not individual rows.

Exercise 4.1: Basic aggregates

- **Task:** Use aggregate functions to compute a single value.
- a) How many employees does the company have?
- b) What is the average salary of all employees?
- c) What is the highest and lowest salary in the company?
- d) What is the total number of hours worked across all projects?

Functions: COUNT, AVG, MAX, MIN, SUM. (5 min)

Exercise 4.2: GROUP BY

- **Task:** Use GROUP BY to compute one value per group.
- a) For each department (Dno), count how many employees work there.
- b) For each department, show the average salary.
- c) For each project (Pno), show the total hours worked.

Every non-aggregate column in SELECT must also be in GROUP BY. (7 min)

Exercise 4.3: HAVING

- **Task:** Use HAVING to filter groups (not rows).
- a) Show departments that have MORE THAN 3 employees.
- b) Show projects where the total hours worked is greater than 50.
- c) Show departments whose average salary is above 30000.

WHERE filters rows, HAVING filters groups (after aggregation). (7 min)

Exercise 4.4: GROUP BY with JOIN

- **Task:** Combine joins with grouping.
- a) For each department NAME (not Dno), show employee count and average salary.
- b) For each project NAME and location, show the total hours worked.
- c) For each manager's first/last name, show how many employees they directly supervise.

Join first to get names, then GROUP BY the names. (8 min)

Exercise 4.5: Nested queries (subqueries)

- **Task:** Use a query inside another query.
- a) Find employees who earn MORE than the average salary.
- b) Find employees who work in the SAME department as John Smith.
- c) Find employees who do NOT have any dependents.
- d) Find the project(s) with the HIGHEST total hours worked.

Operators: IN, NOT IN, EXISTS, comparison with subquery. (10 min)

Submission

- **File format:** one .sql file containing all 21 exercises.
- **File naming:** BIT31_FirstName_LastName_Exercises_Pack.sql

- **Format inside the file:**

```
-- Exercise 1.1 (a)
```

```
INSERT INTO ...;
```

```
-- Exercise 1.1 (b)
```

```
INSERT INTO ...;
```

- **Submit via:** Microsoft Teams — assignments channel.